

AI Automation with n8n

Lesson 7: Deploying and Monitoring Workflows

Deploying and Monitoring Workflows - Study Notes

Key Concepts

1. **Cloud deployment options for n8n** – self hosted (Docker, Kubernetes, VPS) vs. n8n.cloud managed service.
2. **Workflow scheduling** – using the built in Cron node or the “Schedule” trigger to run workflows automatically.
3. **Execution logs & audit trail** – where to find run history, error details, and how to filter/search logs.
4. **Environment variables & secrets** – keeping credentials safe when moving a workflow from local to cloud.
5. **Scaling & reliability basics** – adding a process manager (PM2, Docker compose) and monitoring resource usage.

Summary

Deploying n8n to the cloud lets you run automations 24/7, share them with teammates, and free your local machine from long running processes. The simplest route is to use the official **n8n Docker image** on a VPS, a cloud hosted container service (e.g., Render, Railway, Fly.io), or the fully managed **n8n.cloud** platform. Whichever route you choose, you'll need to configure a persistent data volume (or use the built in SQLite/PostgreSQL DB) so that workflow definitions survive container restarts.

Once the instance is online, you can schedule workflows using the **Cron node** (for complex schedules) or the **Schedule trigger** (simple interval based runs). n8n records every execution in its internal database, and the UI provides a **Execution List** where you can view success/failure status, execution time, input/output data, and error stacks. Filtering by workflow, date range, or status helps you quickly locate problematic runs. For production use, it's good practice to export logs to an external system (e.g., Loki, Papertrail) and to set up alerts for failed executions.

Important Points

- **Choose the right hosting model**
 - **n8n.cloud**: fastest start, automatic backups, limited customisation.
 - **Docker on VPS**: full control, can attach a dedicated PostgreSQL DB, need to manage updates and backups.
 - **Persist data** – mount a volume for `/home/node/.n8n`` (or set `DB_TYPE=postgresdb`` with external DB) to keep workflows after red deploys.
 - **Secure secrets** – use environment variables (`N8N_BASIC_AUTH_USER``, `N8N_ENCRYPTION_KEY``, API keys) or the built in **Credentials** store; never hard code them in workflow nodes.
 - **Scheduling**

- ***Cron node***: full cron syntax (`*/15 * * * *`), runs inside the workflow.
- ***Schedule trigger***: “Every X minutes/hours/days”, easier for non technical users.
 - ****Viewing logs****
- UI !' ****Executions**** !' click a run !' ****Execution Details**** tab.
- Use the ****Filter**** bar to narrow by status, date, or workflow name.
- For deeper debugging, enable ****“Save Execution Data”**** (full input/output) in ****Settings !' Execution****.
 - ****Monitoring & scaling****
- Run n8n behind a process manager (PM2, Docker compose) to auto restart on failure.
- Track CPU/Memory; if you hit limits, consider scaling horizontally (multiple containers) or moving to a larger instance.
 - ****Backup strategy**** – schedule a daily dump of the SQLite file or PostgreSQL DB; store it in object storage (S3, GCS).

Examples

Example 1 – Deploy n8n with Docker on a cheap cloud VPS

```
```bash
```

## 1. Create a persistent volume

```
docker volume create n8n-data
```

## 2. Run the container (replace <YOUR\_SUBDOMAIN> with your domain)

```
docker run -d \
 --name n8n \
 -p 5678:5678 \
 -v n8n-data:/home/node/.n8n \
 -e N8N_BASIC_AUTH_ACTIVE=true \
 -e N8N_BASIC_AUTH_USER=admin \
 -e N8N_BASIC_AUTH_PASSWORD=SuperSecret123 \
 -e N8N_HOST=<YOUR_SUBDOMAIN>.example.com \
 -e N8N_PORT=5678 \
 -e NODE_ENV=production \
 n8nio/n8n:latest
```

```
```
```

- The volume keeps all workflows safe if the container restarts.
- Basic auth protects the UI; for production use, add HTTPS via a reverse proxy (Caddy, Nginx).

Example 2 – Schedule a daily CSV export and monitor it

1. ****Workflow design****

- ****Trigger****: ***Schedule*** !' “Every day at 02:00”.
- ****Node****: ***Google Sheets !' Read Rows*** (fetch data).

- **Node**: `*CSV !` Convert to CSV*.
 - **Node**: `*FTP !` Upload* (store file on a server).
2. **Enable full logging** (Settings ! Execution ! "Save Execution Data" ! "Full").
 3. **Check execution**
 - UI ! **Executions** ! filter "Status = Failed".
 - Click a failed run ! view the error stack (e.g., authentication error).
 4. **Set up an alert** (optional)
 - Add a **Webhook** node at the end of the workflow that posts to a Slack channel when ``{{ $json["error"] }}` exists.

Quick Review

1. What are two main ways to run n8n in the cloud, and what is a key advantage of each?
2. How does the **Cron** node differ from the **Schedule** trigger?
3. Where does n8n store workflow definitions and execution history by default, and how can you make this data persistent?
4. Name three pieces of information you can view in the Execution Details panel.
5. What is a recommended practice for handling API keys or passwords when moving a workflow from local to a cloud instance?

Further Reading

- **n8n Documentation – Deployment** – detailed guides for Docker, Kubernetes, and n8n.cloud.
- **Environment Variables & Secrets Management** – best practices for securing credentials.
- **Process Managers (PM2, Docker compose) & Health Checks** – keeping a production instance resilient.
- **External Log Aggregation** – sending n8n logs to Loki, Papertrail, or CloudWatch.
- **Scaling n8n** – using a PostgreSQL database and multiple worker containers for high throughput workloads.

